



SNOWLEDGE

Developing the Application

8.12.2024

CONTENTS

1.INTRODUCTION	3
2.APPLICATION INFORMATION FOR STAKEHOLDERS	4
3.BEST PRACTICES	5
3.1 Naming practices.....	5
3.2 Comments in code	5
3.3 Changes to existing code	6
3.4 Documentation and class diagram maintenance	6
4.PROGRAM STRUCTURE	8
4.1 Development tools.....	8
4.2 Frontend tools	8
4.3 Backend tools.....	8
4.4 Formatters	9
5.PROJECT DIRECTORY STRUCTURE	10
5.1 Overview	10
5.2 Mobile directory structure	10
5.3 Web directory structure	12
6.GUIDES	13
6.1 Getting started.....	13
6.2 Running the Website	15
6.3 Restarting the Server	16
6.4 Mobile Application	16
7.ISSUES.....	17
7.1 Dependencies/packages.....	17
7.1.1 Description of the issue	17
7.1.2 What was done.....	17
7.1.3 Next steps	18
7.2 Rescue service.....	18
7.2.1 Description of the Issue.....	18
7.2.2 What Was Done	19
7.2.3 Quick Reference Table for Snowledge Deployment	20
7.2.4 What is PM2?	21
7.2.5 How is PM2 Used in Snowledge Server?.....	21
7.2.6 Example Usage in Snowledge	21
7.2.7 Next Steps for the Next Group	22
7.2.8 Additional Suggestions.....	22
8.FINALLY	23

1. INTRODUCTION

The purpose of this document is to collect the most necessary information about developing the Snowledge application into one easily accessible place. It is designed to assist new developers in understanding the project and maintaining an organized codebase. For stakeholders, such as the customer Pallaksen Pöllöt, this document provides information about the development process and foundational aspects of the application. This document should be updated as needed for it to stay relevant and useful.

Chapter 6 Issues is an up-to-date list of the most crucial issues in the Snowledge development. Attempts to solve respective issues are listed there so that the same approaches won't be tried multiple times by different developers. Also potential but not yet tried approaches are listed there.

2. APPLICATION INFORMATION FOR STAKEHOLDERS

The codebase is currently divided into two parts, the iOS and Android applications, and rescue website (lumisovellus.fi/rescue) are in Flutter. The lumisovellus.fi site is written in JavaScript in a separate file. In the future, it is advisable to focus on moving everything to Flutter, when all imminent issues are fixed, as maintaining multiple technologies is a challenge. In addition, Flutter is a more modern platform and offers a lot more flexibility for development in the future, if the program is designed and programmed well from the start.

The development of all the Flutter versions has been hindered by the use of external pieces of code called packages (more detail in chp 6). In short, these packages have not been updated by their makers, and in some cases have been discontinued. There are also newer versions of them available, but since the code has not been updated regularly, the process of upgrading now is very slow. When any code is run that has something from these packages that does not exist anymore, the whole program does not work.

In the next rotation of the development cycle, it is advisable to look for teams with at least some previous experience with Flutter if possible, as fixing the issues in the Flutter packages will otherwise take more time as the group learns the platform. It is not necessary to have a whole team of experts, just one or two advisors in the group would speed up the learning process a lot.

This document serves as a roadmap for understanding the application, maintaining a well-organized codebase, and ensuring that both developers and the customer stay informed about the progress and goals of the project. By documenting best practices, lessons learned, and solutions to common challenges, this guide helps future developers avoid unnecessary setbacks and maintain a high standard of quality. It also fosters transparency and collaboration, ensuring that the customer, Pallaksen Pöllöt, remains updated and confident in the development process.

3. BEST PRACTICES

3.1 Naming practices

All functions and classes are named

- According to their function
- Clearly without unjustified abbreviations or numbers
- With capital first letter as separator between words
- Function names start with lower case first letter, class names start with an uppercase letter
- Global variables (variable that is used by multiple functions) are written in the upper case

Table 1: Examples of use of the naming practices.

Class	SomeClass
Function	someFunction
Global variable	VARIABLE
Variable	variable

3.2 Comments in code

Before each function there needs to be a comment that states:

Short description of what the function does

parameter: What parameters does the function take (name and type)

return: What does the function return

Any *difficult* parts in the code should be commented on to ensure that the code stays understandable. However, the primary means to ensure clarity of code is **simple and clear code structure and naming practices**.

3.3 Changes to existing code

If a function or a class is expanded or otherwise changed, **comments in the code need to be updated** accordingly.

Do not make changes into a class or structure, unless you understand the design principle behind it. It is advisable to do this only after inspecting the structure of the program and updating parts with good design principles in mind. If no documentation exists for the part you are updating, make some. This is an issue most visible in the current website code – no clear structure exists, as classes and components have been constantly changed over time without thought, and have lost their original meaning, if they had any to begin with.

Renaming the class or a function after changes are made to it needs to be considered.

Under no circumstances should you use Flutter packages that are made by undependable publishers, ie non pub.dev verified devs. Even in cases of larger publishers, make sure that the package you decide to use will be maintained in the future. Ignoring this will cause massive pains for future developers, who may have to rewrite the whole program if poor decisions are made here. As this is a project made by and for student groups over a long period of time, do not make “quick fixes” that may fix something fast now, but may cause massive problems for future groups.

3.4 Documentation and class diagram maintenance

There is a drawio class diagram located in the Github repository Snowledge-Web-App: [documentation/class diagram](#)

This was made by the Autumn 2024 group as the initial plan was to move the snowledge website into Flutter to decrease the number of technologies in use in the project. We were unable to even start the process apart from the class diagram for the different parts of the new application, as the issues specified in the Issues section stopped all progress. This diagram should be taken into consideration if similar plans come up in the future. **Update the class diagram** after making changes to the program, do not create duplicates. The old version is not needed after changes have been made.

Update this document after any changes on topics covered here.

4. PROGRAM STRUCTURE

4.1 Development tools

This page provides a detailed overview of all the tools and technologies used throughout the project for efficient development and management

4.2 Frontend tools

The following tools and technologies are employed for frontend development, ensuring an intuitive and responsive user interface across platforms:

1. Web Frontend:

- [React.js](#): A JavaScript library for building interactive user interfaces with a component-based architecture.
- [Material-UI](#): A popular React component library that provides pre-styled components adhering to Google's Material Design guidelines, ensuring consistency and usability.

Allowed practices:

- Customizing Material-UI themes for brand consistency.
- Modular and reusable component design.

2. Mobile Frontend:

- [Flutter](#): A cross-platform UI toolkit used to create native applications for both Android and iOS platforms from a single codebase.

Allowed practices:

- Use of third-party Flutter packages (subject to approval).
- Adhering to Flutter's best practices for responsive design and performance optimization.

4.3 Backend tools

The backend development leverages the following tools and frameworks to handle server-side logic, database management, and API integration:

Web Backend:

1. [Node.js](#): A runtime environment that allows JavaScript to be executed on the server side, suitable for building scalable network applications.
2. [MySQL](#): A relational database management system for storing structured data.

Allowed practices:

- Usage of popular Node.js libraries (e.g., Express.js) for simplified routing middleware.
- Writing optimized SQL queries for efficient data retrieval.

Mobile Backend:

1. [Flask](#): A lightweight Python framework for building robust APIs and microservices.
2. [SQLite](#): A lightweight, file-based database system suitable for mobile application needs.

Allowed practices:

- Securely handling user data and sensitive information.
- Modularizing Flask applications for scalability and maintainability.

API Development:

1. [Postman](#): A powerful API development tool for designing, testing, and documenting APIs.

Allowed practices:

- Collaborative API design with clear documentation.
- Using Postman's testing suite to automate API response validation.

4.4 Formatters

1. To maintain consistency and improve code readability, the following formatters are used across the project:

Web Side Frontend and Backend:

1. [Prettier](#): Automatically formats JavaScript, HTML, CSS, and other web development files to maintain a uniform coding style.

Mobile Backend:

1. [Black](#): An uncompromising Python code formatter that ensures consistent and readable code.

Flutter Code:

1. Dart [formatter](#): Standardizes Dart code to align with Flutter's styling conventions.

Allowed practices:

- Using formatters regularly during development.
- Customizing configuration files for Prettier and Black to suit project guidelines.

5. PROJECT DIRECTORY STRUCTURE

This page outlines the directory structure of the project for clarity and organization.

5.1 Overview

The project comprises two main development directories:

- Mobile snowledge for the mobile versions and the rescue website for the rescue services.
- src for the web version, i.e. Lumisovellus.fi.

```

snowledge
├── /documentation      # Documentation from past groups
├── /Mobile snowledge  # Mobile application
├── /src                # Web application
├── /vagrant            # Vagrant files for VM setup
└── /requirements.txt  # Python dependencies

```

Figure 1: Snowledge Project Structure.

5.2 Mobile directory structure

The Mobile snowledge directory contains the following structure:

- mobile_app for Flutter files (mobile frontend).
- Server with two subdirectories:
 - back_end for server-side code.

Android

The android section of a Flutter application represents the native Android project that interacts with the Flutter framework. It contains files and directories necessary for integrating Flutter with Android, configuring the Android-specific parts of the app, and building the Android app.

1. **.gradle** This folder contains cached files and metadata used by the Gradle build system. It is automatically generated and updated when you run Gradle tasks. It includes information about dependencies, compiled classes, and other build artifacts.
2. **.idea** This folder contains configuration files specific to IntelliJ IDEA or Android Studio. These files store IDE-specific settings such as module configurations, editor preferences, and project metadata.
3. **app**

The app folder contains the main Android application files for the project. Here's a breakdown of its components:

- src/main/AndroidManifest.xml: Defines essential app information like package name, permissions, app components (activities, services), and metadata for integration with Flutter.
- src/main/res: Contains resource files like layouts, images, strings, colors, and themes used in the Android app.

- `src/main/java`: Contains Java (or Kotlin) code for the Android portion of the app. For Flutter, this typically includes a `MainActivity.java` or `MainActivity.kt` file, which serves as the entry point for the Android side of the app.
 - `build.gradle`: The module-level Gradle configuration file. It specifies dependencies, build configurations, and plugin integrations for the Android app.
4. **Gradle/wrapper**: This folder contains files to configure the Gradle Wrapper:
 - **gradle-wrapper.properties**: Specifies the Gradle version and the distribution URL to download the correct version automatically.
 - **gradlew / gradlew.bat**: Script files to run Gradle tasks without needing a pre-installed Gradle on your system. `gradlew` is for Unix-based systems, and `gradlew.bat` is for Windows.
 5. **gradlew / gradlew.bat**
These are shell and batch scripts used to execute Gradle commands via the Gradle Wrapper. Using these ensures the project builds with the exact Gradle version specified in `gradle-wrapper.properties`.
 6. **local.properties**
This file contains local machine-specific configurations, such as the path to the Android SDK
 7. **build**
The build folder is generated during the build process and contains compiled code, intermediate files, and other artifacts. It can be safely deleted and regenerated by running the build again.
 8. **Flutter Plugins Files**
 - **.flutter-plugins**: Tracks Flutter plugins added to the project. It maps plugin names to their locations in the project.
 - **.flutter-plugins-dependencies**: Contains dependency information for the plugins, used during builds.
 9. **Flutter Integration**
The Android section in a Flutter project is set up to integrate Flutter modules into the native Android environment:
 - **Flutter Gradle Plugin**: Configured in `build.gradle` files (both project-level and module-level) to link the Dart code with the Android app.
 - **Generated Files**: When building the app, files like `GeneratedPluginRegistrant.java` are created to register plugins used in the Flutter application.
 10. **build.gradle**
There are typically two `build.gradle` files:
 - **Project-level build.gradle**: Configures build settings for the entire project, like the Gradle plugin version and repositories.
 - **Module-level build.gradle**: Located in the app directory, this file specifies dependencies and build settings for the Android app module.

- front_end for the admin dashboard UI.

```
./Mobile snowledge
├─ /mobile_app      # Flutter files
└─ /Server
   └─ /back_end     # Mobile server files
      └─ /front_end  # Admin dashboard UI
```

Figure 2: Snowledge Mobile Project structure.

5.3 Web directory structure

1. The src directory contains the web application files, including the frontend, backend, and database resources.

```
./src
├─ /map-app        # Web frontend (React & Cypress)
│  └─ /cypress     # Test files
│     └─ /src      # React files for UI
├─ /routers        # Backend routers
├─ /sql            # Map database (SQL)
└─ /app.js         # Express server
```

Figure 3: Web Directory Structure

6. GUIDES

6.1 Getting Started

To begin working on the Snowledge application:

1. Set up the Flutter Development Environment:

Windows:

Install Flutter SDK

- Download Flutter SDK from the official website: flutter.dev.
- Extract the .zip file to a desired location (e.g., C:\src\flutter).

Add Flutter to PATH

- Open "Environment Variables" (Search "Environment Variables" in the Start Menu).
- Under "System variables," find the Path variable and edit it.
- Add the Flutter SDK bin folder path (e.g., C:\src\flutter\bin).

Install Android Studio

- Download and install [Android Studio](#).
- Install the Flutter and Dart plugins:
 - Open Android Studio > Preferences > Plugins > Search for "Flutter" > Install.
- Configure the Android emulator:
 - Open Android Studio > Virtual Device Manager > Create Virtual Device.

Install Visual Studio

- Download and install [Visual Studio](#).
- During installation, select the **"Desktop development with C++"** workload.

Install Additional Dependencies

- Install choco.
- Install required tools with Chocolatey

Run Flutter Doctor

- Open a new terminal and run: **flutter doctor**
- Follow the instructions to resolve any issues if you have any.

macOS:

Install Flutter SDK

- Download Flutter SDK from the official website: flutter.dev.
- Extract the .zip file and move the flutter folder to your desired location (e.g., /Users/your-username/flutter).

Add Flutter to PATH

- Open the terminal and edit your shell configuration file
 - For zsh (default shell on macOS): nano ~/.zshrc
- Add the following line:
 - export PATH="\$PATH:/Users/your-username/flutter/bin"
- Save the file and apply changes:
 - source ~/.zshrc

Install Xcode

- Download and install Xcode from Mac App Store

Install Homebrew

- Run:
 - /bin/bash -c "\$(curl -fsSL <https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh>)"

Install Additional Dependencies

- Install CocoaPods (required for iOS development):
 - sudo gem install cocoapods

Enable iOS Development

- Run:
 - flutter doctor --android-licenses
- Set up a physical device or simulator in Xcode.

Install Android Studio

- Download and install [Android Studio](#).
- Install the Flutter and Dart plugins:
 - Open Android Studio > Preferences > Plugins > Search for "Flutter" > Install.
- Configure the Android emulator:

- Open Android Studio > Virtual Device Manager > Create Virtual Device.

2. Set up the Python Virtual Environment:

- Install Python
- Install Virtual Environment Module
 - `pip install virtualenv`
- If `pip` is not available, install it:
 - `python -m ensurepip --upgrade`
- Navigate to project directory and create virtual environment
 - `python -m venv venv`
- Activate the virtual environment
 - For macOS/Linux:
 - `source venv/bin/activate`
 - For windows:
 - `venv\Scripts\activate`
- Install Flask

3. **Clone the Repository:** Use `git clone` to download the codebase and navigate to the respective folders for frontend and backend development.

4. Verify Setup:

- Run **flutter doctor** to ensure the Flutter environment is correctly set up.
- Activate the Python virtual environment and install required packages with **pip install -r requirements.txt**.
- Run the backend server with `python server.py`.

6.2 Running The Website

The mobile version of Snowledge is developed using Flutter, and the backend uses Flask.

1. Frontend:

- Use `flutter run` to build and run the Flutter application on an emulator or a physical device.

2. Backend:

- Activate the Python virtual environment and start the backend server using `python server.py`.
- Access the website locally by navigating to the server's provided IP address in your browser.

6.3 Restarting The Server

Restarting the server can solve some ambiguous problems or unexpected crashes.

1. Go to tavu.io
2. Sign in
3. Open terminal
4. Enter credentials if needed
5. Write "sudo reboot"

6.4 Mobile Application

For the mobile application to run, make sure you have installed all necessary extensions in Visual Studio Code such as emulators for the phones. **WARNING! Do not allow Visual Studio Code to make any automatic upgrades to the libraries!** This will upgrade to the newest available versions without checking for interoperability between libraries. More information on this can be found in the section "Issues / Libraries".

More specific instructions for running the application can be found in the Readme.md files of the app.

7. ISSUES

Here are the most important issues Autumn 2024 group faced. Included is a description of the issue, what was done, and recommended next steps.

7.1 Dependencies/packages

7.1.1 Description of the issue

The biggest issue facing the development of the current app is the use of deprecated packages. In short, this means that some of the packages that a past team decided to use in their development of the Flutter application have been [removed](#), [not updated](#), [have gone through dramatic changes in syntax](#) (v 3.0.0 vs 7.0.0) or have conflicts between newest versions, as they may use older versions of the same packages in their implementation. This means that the code cannot be compiled without finding specific settings for versions of the packages and the tools used in compilation, something that we were unable to find. Even if these settings were found, they ought not to be used in development, as the technical debt inside the app would only increase as all future developers would be forced to use aging tools and eventually removed packages which would also create conflicts for the use of newer packages.

7.1.2 What was done

We tried running the application with different versions of the tools (Java development kit and different Android/iOS versions), as this was mentioned in previous documentation. This did not yield any results, and it is therefore recommended that this approach is not followed, and focus should be on getting the application to run in newer versions of the packages instead.

We also tried making updates to the existing code file by file to fix errors that came up during runtime, but this is very difficult and slow without any practical results until all errors are fixed. We wanted to make sure that all versions of all tools and the structure of the project itself could be updated and tried moving the files one by one into a new Flutter project to remove any potential for old structures or files, as described next.

We started moving the existing code into a new Flutter project (seen in scratch-branch on Github) to resolve the package issues, but the problems are so pervasive, that we were unable to fix them all in our time. At the time of writing, only the lib file from the original application was moved and attempted to fix without complete success. Therefore, when Android, iOS and web files are moved, we expect the same issues and much more will have to be tackled there, and the time taken to do this may vary. However, when one fixes the Android

file, for example, it could be run separately from the other files, giving results before the whole app is fixed.

7.1.3 Next steps

It is very important to update the Flutter app into a state where more recent versions of the packages are used instead. This is an enormous task that can be approached, in our view, in three different ways:

1. Go through the code file by file, get to know the packages being used and update code by fixing errors as they pop up in compilation. (Autumn 2024 tried this, the process is slow but may work)
2. Start building the app from scratch, as at least some features from these packages are so deprecated, that it may be easier to just rewrite code from scratch, rather than trying to wrestle with old structures and methods in code.
3. Combine these two approaches, rewriting only the most difficult files, and making small fixes to less affected ones.

7.2 Rescue service

7.2.1 Description of the Issue

The rescue service, part of the **Snowledge Flutter application**, appears to have been neglected or broken during a past deployment. Attempts to access the rescue website at <https://lumisovellus.fi/rescue> result in a **"404 Not Found"** error.

The server's error.log contains entries resembling the following:

error log can be found under

```
var/nginx/error.log
```

You can then tail using

```
sudo tail -f error.log
```

```
2024/11/11 11:40:28 [error] 830#830: *13540 open() "/home/juuso/snowledge_rescue/rescue" failed (2: No such file or directory),
client: [REDACTED], server: pallas.lumisovellus.fi, request: "GET /rescue HTTP/1.1", host: "lumisovellus.fi"
2024/11/11 11:40:31 [error] 830#830: *13540 open() "/home/juuso/snowledge_rescue/rescue" failed (2: No such file or directory),
client: [REDACTED], server: pallas.lumisovellus.fi, request: "GET /rescue HTTP/1.1", host: "lumisovellus.fi"
```

Figure 4: Error log view

This error suggests that the required resources were previously located at `snowledge_rescue/rescue`, but subsequent updates to the file structure were not reflected in the app or server code, leading to the current issue.

7.2.2 What Was Done

We conducted several troubleshooting steps to identify and resolve the issue:

1. Nginx server update:

- a. Updated the server's NGINX configuration to point to potential new paths where the files might have been relocated.
- b. Attempted to manually create placeholder files and directories at the original location (`snowledge_rescue/rescue`) to test if the system could detect them.
- c. **Result:** No change, and the error persisted.

2. Static File Testing:

- a. Configured a temporary NGINX rule to serve a simple static HTML file at the rescue URL.
- b. **Result:** Verified that NGINX could serve static content, ruling out issues with NGINX's ability to serve files generally.

3. Codebase and Deployment Review:

- a. Searched the source code and deployment scripts for hardcoded paths or misconfigured references to the rescue service.
- b. Identified outdated references pointing to `snowledge_rescue/rescue` but could not locate the missing files in the current repository.

4. Rollback Attempt:

- a. Tried to rollback the server configuration to an older version using available backups, but older files related to the rescue service were not present in these backups.

5. Flutter-Specific Debugging:

- a. Verified that the rescue functionality was properly referenced in `pubspec.yaml` and that all necessary dependencies were present.
- b. Confirmed that no recent Flutter build outputs contained the rescue service's functionality.

6. Some quick references. These commands are available in snowledge wiki in Gitlab.

Quick Reference for server

Task	Command/Step
Update server packages	<code>sudo apt update && sudo apt -y upgrade</code>

Restart server	sudo reboot
Start web app	sudo pm2 start app.js
Check PM2 processes	sudo pm2 list
Start backend services	PM2 commands with --interpreter=/path/to/python3.9
Activate Python env	source /path/to/virtualE/bin/activate
Install npm packages	sudo npm install
Build React app	sudo npm run build
Clone repository	git clone git@gitlab.com:lumisovellus/snowledge.git

7.2.3 Quick Reference Table for Snowledge Deployment

Task	Command/Step
Access Server	ssh juuso@185.87.111.109 (Ask @pallaksenpollot for the password)
Check PM2 Processes	sudo pm2 list
Stop All PM2 Processes	sudo pm2 stop all
Start a PM2 Process	sudo pm2 start <file> --name=<process_name>
Restart a PM2 Process	sudo pm2 restart <process_name>
Save PM2 State	sudo pm2 save
Set PM2 to Autostart	sudo pm2 startup
View PM2 Logs	sudo tail -f /home/juuso/.pm2/logs/<log_file>
Navigate to React App Directory	cd ~/snowledge/src/map-app
Clean Dependencies	rm -rf node_modules package-lock.json
Install NPM Packages	sudo npm install
Build React App	sudo npm run build
Start React Client	sudo pm2 start app.js --name=react_client
Navigate to Backend Directory	cd ~/snowledge/Mobile\ snowledge/Server/back_end
Activate Python Virtual Env	source ~/snowledge_2023/snowledge/virtualE/bin/activate
Start Backend Processes	sudo pm2 start server.py --interpreter=/path/to/python3.9 --name=backend_server
""""""	sudo pm2 start _rescue_management_api.py --interpreter=/path/to/python3.9 --name=rescue_api
""""""	sudo pm2 start _app.py --interpreter=/path/to/python3.9 --name=app_api
Install Missing Python Packages	pip install <missing_package>
Update Server Packages	sudo apt update && sudo apt -y upgrade
Restart Server	sudo reboot
Verify .env Files	Ensure proper production configuration in:
	- ./Mobile snowledge/mobile_app/.env
	- ./Mobile snowledge/Server/back_end/.env
Clone Repository	git clone git@gitlab.com:lumisovellus/snowledge.git
Pull Latest Code	git pull origin master

7.2.4 What is PM2?

PM2 is a production-ready process manager for Node.js applications. It simplifies the management of application processes, enabling tasks such as starting, stopping, restarting, monitoring, and auto-restarting processes. PM2 also supports load balancing, log management, and automatic startup configuration, making it ideal for running server applications in production environments.

7.2.5 How is PM2 Used in Snowledge Server?

In the Snowledge project, PM2 is used to manage both the **backend services** and the **React client**. Its primary roles include:

1. **Process Management:**
PM2 ensures that all backend Python scripts and the React app are running as background processes. This allows the server to remain operational even if terminal sessions are closed.
2. **Monitoring:**
PM2 provides visibility into the status of running processes (e.g., uptime, restart counts) using the `pm2 list` command.
3. **Auto-Restart:**
PM2 automatically restarts processes that crash, ensuring high availability.
4. **Startup Configuration:**
PM2 can save the current running processes and configure them to start automatically when the server reboots (`pm2 save` and `pm2 startup`).
5. **Log Management:**
PM2 captures logs for each process, making it easier to debug issues when they arise.

7.2.6 Example Usage in Snowledge

- **Starting Processes:**

PM2 is used to start backend Python scripts (`server.py`, `_rescue_management_api.py`, `_app.py`) and the React app (`app.js`).

- **Checking Status:**

Developers can monitor process uptime, restarts, and logs using:

```
sudo pm2 list
```

Auto-Startup:

PM2 ensures the app is restarted automatically after a server reboot, configured using:

```
sudo pm2 startup
```

```
sudo pm2 save
```

PM2 plays a critical role in maintaining the stability and reliability of the Snowledge application in production.

7.2.7 Next Steps for the Next Group

To restore the rescue service, the following actions are recommended:

1. **Recover Missing Files or Equivalent:**
 - a. Check all previous deployments, repositories, or team archives for the original snowledge_rescue/rescue files.
 - b. Determine if equivalent files or functionality exist in the current app structure that could replace the missing resources.
2. **Update Server Configuration:**
 - a. Modify the NGINX configuration to correctly reference the new location of the rescue service files or their equivalent.
 - b. Test the configuration with the `nginx -t` command to ensure no syntax errors.
3. **Debug the Flutter App:**
 - a. Analyze whether the rescue service functionality was removed from the Flutter application during an earlier refactor or update.
 - b. Ensure the app references align with the server's expectations and rebuild the app if necessary.
4. **Implement Error Simulation:**
 - a. Use tools like curl or browser developer tools to simulate requests to the rescue URL and identify discrepancies in response headers or server routing.
5. **Set Up Logging for NGINX and Backend Services:**
 - a. Enhance logging verbosity in NGINX and backend service configurations to identify why the requests fail (e.g., routing, file access).
6. **Create a Staging Environment for Testing:**
 - a. Test all fixes in a separate staging environment before applying changes to production.

7.2.8 Additional Suggestions

- **Future-Proofing:**

Implement CI/CD pipelines with automated deployment and error detection to catch similar issues early.

- **Documentation:**

Maintain detailed documentation of file structure changes and service updates to help future teams avoid such challenges.

8. FINALLY

The existing code base does not follow all these instructions. However, keeping these in mind is important so that the code base can evolve to a clearer form. This document is just a general idea to get you starting but there are some dependencies issues in the mobile application .Next time whichever group is taking the snowledge application project please first try to run the whole application in your local computer first even before starting doing anything else because it could be high likely that there is some dependency which is outdated with your local environment . And also try to communicate with the founders as much as possible, they are really helpful, and they will then tell you what to do.

Remember to update this document if needed.